PROGRAMACIÓN CONCURRENTE EN PYTHON

Diseño del Servidor de Descargas Concurrente

Análisis de recursos compartidos, representaciones de datos, sección crítica y mecanismos de sincronización

Integrantes: Anthony Martinez, Jorgue Medina

01. CONTEXTUALIZACIÓN

Contexto del Servidor y Actividades Concurrentes

Peticiones Multicliente

Múltiples clientes realizan solicitudes de descarga en paralelo hacia un servidor con ancho de banda y conexiones restringidas.

Ejecución mediante Hilos

Cada cliente se representa como un hilo independiente (`DownloadWorker`) ejecutando validación, transferencia y registro final.



02. CONCEPTO CLAVE

El Recurso Compartido y Conflictos de Acceso

¿Qué representa?

La capacidad total del pool de descarga del servidor y el registro acumulado del ancho de banda servido.

¿Quiénes lo utilizan?

Todos los hilos de clientes concurrentes (`DownloadWorker`) que compiten por un cupo de descarga libre.

¿Por qué hay conflicto?

El acceso concurrente no coordinado causa condiciones de carrera, sobrepasando la capacidad y corrompiendo contadores.

03. ALTERNATIVAS DE DISEÑO

Modelado del Recurso Compartido

OPCIÓN A

Tipo Básico (Contador)

Conexiones Activas:	int
Tráfico Acumulado:	int

Solo almacena magnitudes numéricas. No guarda identidad ni metadata del cliente.

OPCIÓN B

Tipo Complejo (Diccionario)

Mapa de Slots (0..N):	dict
Metadata (Cliente, Archivo):	sub-dict

Permite monitoreo detallado, estado por canal y trazabilidad completa.

04. ARQUITECTURA DE CLASES

Diagrama de Clases UML del Sistema



05. FLUTO DE EJECUCIÓN CRÍTICO

Flujo de la Sección Crítica y Riesgos

1. **Consulta de Disponibilidad**
   Verificar si hay espacio o slot disponible en el servidor.

2. **Validación de Regla de Negocio**
   Evaluar si no se sobrepasa el límite máximo permitido.

3. **Modificación del Recurso**
   Asignar el slot y actualizar el acumulador de bytes.

⚠ Sobreescritura de Slot

Dos hilos leen la misma posición como libre al mismo tiempo, resultando en un cliente desconectado o ignorado.

⚠ Pérdida en Tráfico Acumulado

Escrituras concurrentes no atómicas provocan que se pierda la suma del ancho de banda servido por uno de los hilos.

06. PROTECCIÓN Y SINCRONIZACIÓN

Sincronización mediante Mutex (Exclusión Mutua)

Adquisición del Cerroj
Al ingresar a `request_slot()` o `release_slot()`

Ejecución Atómica
Un solo hilo manipula el diccionario y el tráfico acumulado

Liberación del Cerrojo
Garantizado automáticamente al salir de la función

¿Por qué Mutex y no Semáforo?
El semáforo solo controla contadores de cupo, pero no protege las modificaciones internas de la estructura compleja (slots y métricas).

Punto de Adquisición y Liberación
Se adquiere inmediatamente antes de consultar los slots y se libera automáticamente justo al terminar de actualizar el estado.

04. ARQUITECTURA DE CÓDIGO

Clases Propuestas e Instancia Compartida

```
# Instancia única compartida
server = DownloadServerManager(max_cap=5)

class DownloadWorker(threading.Thread):
  def __init__(self, cliente_id, server):
    super().__init__()
    self.server = server # Misma referencia
    self.cliente_id = cliente_id

  def run(self):
    slot = self.server.request_slot(self.name)
    if slot =/= -1:
      self.server.release_slot(slot, 1024)
```

DownloadServerManager

Administra el estado global, valida límites y acumula métricas de tráfico.

DownloadWorker (Thread)

Recibe la referencia a la misma instancia de `server` en su constructor.

05. ANÁLISIS DE RIESGOS

Sección Crítica e Inconsistencias

```
# SECCIÓN CRÍTICA EN REQUEST_SLOT:
for slot_id, data in self.slots.items(): # 1. Consulta
  if data is None: # 2. Validación
    self.slots[slot_id] = info_hilo # 3. Modificación
    return slot_id

# SECCIÓN CRÍTICA EN RELEASE_SLOT:
self.slots[slot_id] = None # Modificación
self.total_bytes += bytes_trans # Modificación
```

Riesgo 1: Sobreescritura de Slot

Dos hilos ven el mismo slot libre a la vez y se sobreescriben, perdiendo una descarga activa.

Riesgo 2: Condición de Carrera en Bytes

Lectura/Escritura simultánea en `total_bytes` causa pérdida de registros de tráfico acumulado.

06. PROTECCIÓN Y SINCRONIZACIÓN

Protección con Cerrojo Mutex (threading.Lock)

```
class DownloadServerManager:
  def __init__(self):
    self._lock = threading.Lock()

  def request_slot(self, info_hilo):
    with self._lock: # Adquiere automáticamente
      # Operaciones atómicas en slots
      return slot_asignado
    # Libera automáticamente al salir
```

Justificación de Elección

Se elige Mutex (Lock) frente a Semáforo porque no solo controlamos un contador, sino que modificamos una estructura compleja de slots de manera atómica.

Adquisición y Liberación

El gestor de contexto `with self._lock:` asegura la liberación estricta del cerrojo incluso si ocurren excepciones.

07. VERDICTO FINAL

Elección de la Representación Compleja

Decisión: Estructura Tipo Complejo (Diccionario)

Justificación: Aunque el tipo básico requiere menos líneas, el **tipo complejo** proporciona auditabilidad completa, trazabilidad de clientes por slot (ID de cliente, archivo, tamaño MB) y monitoreo en tiempo real, indispensable en arquitecturas de servidores en producción.

CONCLUSIÓN DEL ESTUDIO

Diseño Concurrente Válido y Protegido

La combinación del tipo complejo con exclusión mutua asegura la máxima visibilidad del sistema sin comprometer la integridad de la memoria.

Fin de la Presentación

Image Sources



https://cdn.shopify.com/s/files/1/0736/7566/9719/files/cisco-smart-switches-sdwan-ai-datacenter.jpg

Source: network-switch.com